[3] Project Setup Guide

Project: Eventura (powered by Clientura) — Enterprise Event & Hackathon Management Platform
Document Version: 1.0.0
Target Environments: Local Development, Staging, and Production
Date: October 29, 2026
Audience: DevOps Engineers, Full-Stack Developers, QA Engineers, System Administrators

1. Introduction

This guide provides end-to-end, reproducible setup instructions to install, configure, migrate, and run **Eventura** across any development workstation or cloud server environment (Windows, macOS, Ubuntu/Debian Linux).

2. System Prerequisites & Hardware Specifications

2.1 Hardware Requirements

- **Minimum**: 4 CPU cores, 8 GB RAM, 10 GB free disk space.
- **Recommended**: 8 CPU cores, 16 GB RAM, NVMe SSD storage.

2.2 Software Requirements

Software Dependency	Required Version	Verification Command	Notes
Node.js	`⩾ 18.0.0` (v20.x or v22.x LTS recommended)	`node -v`	JavaScript runtime
npm or **bun**	npm `⩾ 9.0.0` / bun `⩾ 1.1.0`	`npm -v` / `bun -v`	Package manager
PostgreSQL	`⩾ 14.0` (v15+ recommended)	`psql --version`	Primary relational database
Git	`⩾ 2.30.0`	`git --version`	Source control
Google Chrome / Chromium	Latest	`chrome --version`	Required for headless PDF generation
Redis *(Optional)*	`⩾ 7.0`	`redis-cli ping`	Distributed caching & rate limiting

3. Step-by-Step Installation

Step 3.1: Clone the Repository

Clone the codebase to your chosen local directory:

```
git clone <repository-url> event_management
cd event_management
```

Step 3.2: Install Node Dependencies

Install all required production and development dependencies using npm or bun:

```
# Using npm
npm install

# Alternatively using bun
bun install
```

Step 3.3: Environment Configuration

Duplicate the `.env.example` file to create your active `.env`:

```
# Windows PowerShell
Copy-Item .env.example .env

# macOS / Linux
cp .env.example .env
```

Open `.env` and verify the following environment variable definitions:

Variable Name	Required	Default / Example Value	Description
`NODE_ENV`	Yes	`development`	Set to `production` in staging/production
`PORT`	Yes	`3000`	Port for the Express REST API backend server
`DATABASE_URL`	Yes	`postgresql://postgres:postgres@localhost:5432/eventura_db?schema=public`	PostgreSQL connection string
`JWT_SECRET`	Yes	`super-secret-high-entropy-jwt-key-256bit`	Secret key for signing access JWT tokens
`JWT_REFRESH_SECRET`	Yes	`super-secret-refresh-token-key-256bit`	Secret key for signing refresh tokens
`FRONTEND_URL`	Yes	`http://localhost:8080`	URL of the frontend application (for CORS)
`VITE_API_BASE_URL`	Yes	`http://localhost:3000/api/v1`	Backend API base URL for client requests
`STRIPE_SECRET_KEY`	No	`sk_test_...`	Stripe secret key for payment processing
`STRIPE_WEBHOOK_SECRET`	No	`whsec_...`	Stripe webhook signing secret
`GEMINI_API_KEY`	No	`AIzaSy...`	Google Gemini AI key for copilot & validation
`REDIS_URL`	No	`redis://localhost:6379`	Optional Redis cache connection string

---

4. Database Setup, Migrations & Seeding

Step 4.1: Create PostgreSQL Database

Ensure your PostgreSQL server is active, then create the target database:

```
-- Connect via psql
psql -U postgres
CREATE DATABASE eventura_db;
\q
```

Step 4.2: Generate Prisma Client & Run Migrations

Run Prisma client generation and apply the forward-only database migrations:

```
# 1. Generate Prisma Client bindings
npm run db:generate
# (or npx prisma generate)

# 2. Apply all forward migrations to the database
npx prisma migrate deploy

# (For clean development environments, you may also use: npx prisma db push)
```

Step 4.3: Seed Initial Data & RBAC Permissions

Populate the database with default organizations, roles, permissions, administrative accounts, and sample events:

```
# Run master database seeder
npm run db:seed

# Run idempotent reports permissions sync
npm run sync:reports-permissions
# (or npx tsx prisma/scripts/sync-reports-permissions.ts)
```

---

5. Running the Application

Option 5.1: Local Development Mode (Concurrent)

To develop locally with hot module replacement (HMR), run the backend API and frontend Vite servers in separate terminals:

Terminal 1 — Backend Express API (Port 3000):

```
npm run dev:server
# Output: [server] Express API listening on port 3000
# Output: [server] Health check: http://localhost:3000/api/v1/health
```

Terminal 2 — Frontend React + Vite SSR (Port 8080):

```
npm run dev
# Output: [vite] Local: http://localhost:8080/
```

Option 5.2: Production Build & Execution

To compile and execute the production server locally or on a staging server:

```
# Step 1: Compile the frontend and Nitro SSR server bundle
npm run build

# Step 2: Start the production Nitro SSR server
node .output/server/index.mjs

# Step 3: In a separate process, start the production backend API
npm run server
```

---

6. Default Administrative Credentials & Test Accounts

The seeder initializes predefined accounts for testing each persona:

Role / Persona	Email Address	Default Password	Initial MFA State
Sudo / Super Admin	`admin@eventura.dev`	`Admin123!`	TOTP MFA Enabled
Organization Admin	`orgadmin@eventura.dev`	`Admin123!`	TOTP MFA Enabled
Event Manager	`manager@ascent.dev`	`Password123!`	Standard Password
Faculty Coordinator	`faculty@ascent.dev`	`Password123!`	Standard Password
Student Coordinator	`student@ascent.dev`	`Password123!`	Standard Password
Judge / Evaluator	`judge@ascent.dev`	`Password123!`	Standard Password
Mentor	`mentor@ascent.dev`	`Password123!`	Standard Password
Participant	`participant@ascent.dev`	`Password123!`	Standard Password

[NOTE]: > For administrative accounts with TOTP MFA configured in development, recovery codes are generated during enrollment or can be managed in the `UserMfa` table.

---

7. Verification & Health Check Procedure

Once the servers are running, verify system integrity with the following checks:

1. **API Health Endpoint**:

```
curl http://localhost:3000/api/v1/health
# Expected: {"success":true,"message":"Eventura API is healthy"}
```

2. **Automated Smoke Test**:

```
npm run test:smoke
# Expected: Total Checks: 10 | Passed: 10 | Failed: 0
```

3. **Frontend Application**: Navigate to `http://localhost:8080/login` in your web browser. Confirm the login interface and mascot animations load cleanly.

---

8. Troubleshooting & Common Issues

Issue / Error	Likely Cause	Solution
`PrismaClientInitializationError: Can't reach database server`	PostgreSQL is not running or credentials in `DATABASE_URL` are incorrect.	Verify PostgreSQL service is active (`Get-Service postgresql*` or `sudo systemctl status postgresql`) and test credentials with `psql` .
`HTTP 400 MISSING_TENANT`	The HTTP request is missing the required `x-organization-id` header on a tenant-scoped endpoint.	Ensure your frontend or API client includes the `x-organization-id` header matching the active workspace.
`CORS Error: No 'Access-Control-Allow-Origin' header`	`FRONTEND_URL` in backend `.env` does not match the client origin.	Set `FRONTEND_URL="http://localhost:8080"` in `.env` and restart the API server.
`HTTP 403 Forbidden on /reports/*`	The user role lacks `reports.read` or `reports.export` permission.	Run `npm run sync:reports-permissions` to synchronize permissions to database roles.
`Port 3000 or 8080 already in use`	A zombie Node process is occupying the port.	Kill existing processes: `Stop-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess -Force` (Windows) or `kill -9 $(lsof -t -i:3000)` (Linux/macOS).